ASSIGNMENT- 05

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

Subject: Database Management System

Topic: Database Creation and Table Implementation

By
Name: Mohammed Sinan nm
Reg. No.: 2511021240024

Faculty Name: RENITA JOY
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
August 2026

DATABASE CREATION AND TABLE IMPLEMENTATION

1. Introduction

A Database Management System (DBMS) is used to store, organize, retrieve, and manage data efficiently. SQL (Structured Query Language) provides commands for creating databases, defining tables, and establishing relationships between them.

In this assignment, the database for an **Online Food Delivery System**, similar to Swiggy or Zomato, is implemented using SQL Data Definition Language (DDL) commands.

The system manages customers, delivery addresses, restaurants, menu items, orders, order items, delivery agents, deliveries, payments, and customer reviews.

The database consists of ten tables connected through primary keys and foreign keys. These relationships help maintain data integrity and reduce unnecessary duplication.

2. Objectives

The main objectives of this assignment are:

1. To create a database for an Online Food Delivery System.
2. To implement all ten tables using SQL DDL commands.
3. To define primary keys and foreign keys.
4. To establish relationships between the tables.
5. To apply appropriate constraints to maintain data accuracy.
6. To create a structured and reliable relational database.

3. Software Requirements

- **Database Management System:** MySQL
- **Query Language:** SQL
- **Commands Used:** CREATE DATABASE, USE, CREATE TABLE, PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK

4. Database Design

The database contains the following ten tables:

Sl. No.	Table Name	Purpose
1	Customers	Stores customer details.
2	Addresses	Stores customer delivery addresses.
3	Restaurants	Stores restaurant information.
4	MenuItems	Stores restaurant menu items.
5	Orders	Stores customer orders.
6	OrderItems	Stores individual items within an order.
7	DeliveryAgents	Stores delivery agent details.
8	Deliveries	Tracks order deliveries.
9	Payments	Stores payment information.
10	Reviews	Stores customer ratings and feedback.

5. SQL DDL Implementation

5.1 Creating the Database

The CREATE DATABASE command creates a new database, while USE selects it for subsequent operations.

```
CREATE DATABASE FoodDeliveryDB;
```

```
USE FoodDeliveryDB;
```

5.2 Creating the Customers Table

The Customers table stores customer details, including name, email, phone number, password hash, and registration date.

```
CREATE TABLE Customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone_number VARCHAR(15) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    registration_date DATE NOT NULL
);
```

Explanation:

- customer_id uniquely identifies each customer.
- AUTO_INCREMENT automatically generates customer IDs.
- PRIMARY KEY ensures that each customer has a unique identifier.
- UNIQUE prevents duplicate email addresses and phone numbers.
- NOT NULL ensures that required fields contain values.

5.3 Creating the Addresses Table

The Addresses table stores one or more delivery addresses for each customer.

```
CREATE TABLE Addresses (
    address_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    address_line VARCHAR(150) NOT NULL,
    city VARCHAR(50) NOT NULL,
```

```
    state VARCHAR(50) NOT NULL,
    pincode VARCHAR(10) NOT NULL,
    address_type VARCHAR(20) NOT NULL,

    FOREIGN KEY (customer_id)
        REFERENCES Customers(customer_id)
);
```

Relationship: One customer can have multiple delivery addresses (1:M).

The customer_id foreign key connects each address to its owner in the Customers table.

5.4 Creating the Restaurants Table

The Restaurants table stores restaurant names, cuisine types, addresses, phone numbers, ratings, and availability status.

```
CREATE TABLE Restaurants (
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_name VARCHAR(100) NOT NULL,
    cuisine_type VARCHAR(50) NOT NULL,
    address VARCHAR(150) NOT NULL,
    phone_number VARCHAR(15) NOT NULL,
    rating DECIMAL(2,1) DEFAULT 0.0,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,

    CHECK (rating BETWEEN 0 AND 5)
);
```

Explanation:

- restaurant_id uniquely identifies each restaurant.

- DEFAULT 0.0 assigns an initial rating when one is not provided.
- BOOLEAN represents whether the restaurant is active.
- CHECK restricts ratings to values between 0 and 5.

5.5 Creating the MenuItems Table

The MenuItems table stores the food items offered by each restaurant.

```
CREATE TABLE MenuItems (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_id INT NOT NULL,
    item_name VARCHAR(100) NOT NULL,
    description VARCHAR(200),
    price DECIMAL(8,2) NOT NULL,
    category VARCHAR(30) NOT NULL,
    is_available BOOLEAN NOT NULL DEFAULT TRUE,

    FOREIGN KEY (restaurant_id)
        REFERENCES Restaurants(restaurant_id),

    CHECK (price >= 0)
);
```

Relationship: One restaurant can have multiple menu items (1:M).

The restaurant_id foreign key identifies the restaurant that offers each item.

5.6 Creating the Orders Table

The Orders table stores information about orders placed by customers.

```
CREATE TABLE Orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
```

```
    restaurant_id INT NOT NULL,
    address_id INT NOT NULL,
    order_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    order_status VARCHAR(20) NOT NULL DEFAULT 'Placed',
    total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00,

    FOREIGN KEY (customer_id)
        REFERENCES Customers(customer_id),

    FOREIGN KEY (restaurant_id)
        REFERENCES Restaurants(restaurant_id),

    FOREIGN KEY (address_id)
        REFERENCES Addresses(address_id),

    CHECK (order_status IN (
        'Placed',
        'Preparing',
        'Out for Delivery',
        'Delivered',
        'Cancelled'
    )),

    CHECK (total_amount >= 0)
);
```

Relationships:

- One customer can place multiple orders (1:M).
- One restaurant can receive multiple orders (1:M).
- One saved address can be referenced by multiple orders (1:M).

The foreign keys ensure that every order references an existing customer, restaurant, and address.

5.7 Creating the OrderItems Table

The OrderItems table stores the individual food items included in each order.

```
CREATE TABLE OrderItems (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    item_id INT NOT NULL,
    quantity INT NOT NULL,
    price_at_order DECIMAL(8,2) NOT NULL,

    FOREIGN KEY (order_id)
        REFERENCES Orders(order_id),

    FOREIGN KEY (item_id)
        REFERENCES MenuItems(item_id),

    CHECK (quantity > 0),
    CHECK (price_at_order >= 0)
);
```

Relationships:

- One order can contain multiple order items (1:M).
- One menu item can appear in multiple order items (1:M).

The OrderItems table resolves the many-to-many relationship between Orders and MenuItems.

The price_at_order attribute preserves the price charged when the item was ordered, even if its current menu price changes later.

5.8 Creating the DeliveryAgents Table

The DeliveryAgents table stores information about delivery personnel.

```
CREATE TABLE DeliveryAgents (
    agent_id INT AUTO_INCREMENT PRIMARY KEY,
    agent_name VARCHAR(100) NOT NULL,
    phone_number VARCHAR(15) NOT NULL,
    vehicle_number VARCHAR(20) NOT NULL,
    is_available BOOLEAN NOT NULL DEFAULT TRUE
);
```

Explanation:

Each delivery agent has a unique agent_id. The is_available attribute indicates whether the agent is available to accept another delivery.

5.9 Creating the Deliveries Table

The Deliveries table tracks delivery assignments and the progress of each order's delivery.

```
CREATE TABLE Deliveries (
    delivery_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL UNIQUE,
    agent_id INT NOT NULL,
    pickup_time DATETIME,
    delivery_time DATETIME,
    delivery_status VARCHAR(20) NOT NULL DEFAULT 'Assigned',

    FOREIGN KEY (order_id)
```

```
    REFERENCES Orders(order_id),

  FOREIGN KEY (agent_id)
    REFERENCES DeliveryAgents(agent_id),

  CHECK (delivery_status IN (
    'Assigned',
    'Picked Up',
    'Delivered',
    'Failed'
  ))
);
```

Relationships:

- Each order can have at most one delivery record (1:1).
- One delivery agent can handle multiple deliveries over time (1:M).

The UNIQUE constraint on order_id ensures that the same order cannot have multiple delivery records in this design.

5.10 Creating the Payments Table

The Payments table stores payment amounts, methods, statuses, and dates.

```
CREATE TABLE Payments (
  payment_id INT AUTO_INCREMENT PRIMARY KEY,
  order_id INT NOT NULL UNIQUE,
  amount DECIMAL(10,2) NOT NULL,
  payment_method VARCHAR(20) NOT NULL,
  payment_status VARCHAR(20) NOT NULL DEFAULT 'Pending',
  payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (order_id)
        REFERENCES Orders(order_id),

    CHECK (amount >= 0),

    CHECK (payment_method IN (
        'UPI',
        'Card',
        'Net Banking',
        'Cash on Delivery'
    )),

    CHECK (payment_status IN (
        'Pending',
        'Successful',
        'Failed'
    ))
);
```

Relationship: Each order can have at most one payment record (1:1).

The order_id foreign key connects each payment to its corresponding order. The UNIQUE constraint enforces the one-payment-record-per-order design.

5.11 Creating the Reviews Table

The Reviews table stores customer ratings and feedback about restaurants and delivery experiences.

```
CREATE TABLE Reviews (
    review_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    order_id INT NOT NULL UNIQUE,
    customer_id INT NOT NULL,
    restaurant_rating INT NOT NULL,
    delivery_rating INT NOT NULL,
    comment VARCHAR(250),
    review_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (order_id)
        REFERENCES Orders(order_id),

    FOREIGN KEY (customer_id)
        REFERENCES Customers(customer_id),

    CHECK (restaurant_rating BETWEEN 1 AND 5),
    CHECK (delivery_rating BETWEEN 1 AND 5)
);
```

Relationships:

- Each order can have at most one review (1:1).
- One customer can write multiple reviews for different orders (1:M).

The rating constraints ensure that both ratings remain within the range of 1 to 5.

The design should also ensure that the customer writing a review is the customer who placed the corresponding order.

6. Summary of Table Relationships

Parent Table	Child Table	Relationship	Purpose

Customers	Addresses	1:M	A customer can save multiple addresses.
Customers	Orders	1:M	A customer can place multiple orders.
Customers	Reviews	1:M	A customer can review different orders.
Restaurants	MenuItems	1:M	A restaurant can offer multiple food items.
Restaurants	Orders	1:M	A restaurant can receive multiple orders.
Addresses	Orders	1:M	An address can be used for multiple orders.
Orders	OrderItems	1:M	An order can contain multiple items.
MenuItems	OrderItems	1:M	A menu item can appear in multiple orders.
Orders	Deliveries	1:1	An order has at most one delivery record.
DeliveryAgents	Deliveries	1:M	An agent can handle multiple deliveries.
Orders	Payments	1:1	An order has at most one payment record.
Orders	Reviews	1:1	An order can have at most one review.

7. Important SQL DDL Commands Used

The following SQL commands and constraints are used in the implementation:

Command or Constraint	Purpose
CREATE DATABASE	Creates a new database.
USE	Selects the database for subsequent operations.
CREATE TABLE	Creates a table and defines its columns.
PRIMARY KEY	Uniquely identifies each record.
FOREIGN KEY	Establishes a relationship between tables.
AUTO_INCREMENT	Automatically generates numeric identifiers.
NOT NULL	Prevents a column from storing NULL values.
UNIQUE	Prevents duplicate values in a column or key combination.
DEFAULT	Assigns a default value when one is not provided.
CHECK	Validates data against a specified condition.

8. Execution and Verification

To implement the database:

1. Open MySQL Workbench or another MySQL-compatible SQL environment.
2. Execute the CREATE DATABASE command.
3. Select the database using the USE command.
4. Execute the ten CREATE TABLE statements in the order presented.

5. Verify that all tables have been created successfully.

The tables must be created in the correct order because some tables reference other tables through foreign keys.

For example, Customers and Restaurants must exist before Orders can be created. Similarly, Orders and MenuItems must exist before OrderItems can be created.

The following commands can be used to verify the implementation:

```
SHOW DATABASES;
```

```
USE FoodDeliveryDB;
```

```
SHOW TABLES;
```

To inspect the structure of a table, use:

```
DESCRIBE Customers;
```

```
DESCRIBE Orders;
```

```
DESCRIBE OrderItems;
```

The SHOW TABLES command lists the tables in the selected database, while DESCRIBE displays the columns, data types, and key information for a table.

9. Result

The database FoodDeliveryDB is defined with ten tables representing the major components of an Online Food Delivery System.

Primary keys provide unique identification for records, and foreign keys establish relationships between related tables. Additional constraints such as UNIQUE, NOT NULL, DEFAULT, and CHECK help maintain data integrity.

The database structure supports the storage and management of customers, addresses, restaurants, menu items, orders, order items, delivery agents, deliveries, payments, and reviews.

10. Conclusion

This assignment demonstrates the creation and implementation of a relational database using SQL DDL commands. The Online Food Delivery System was designed with ten tables, each serving a specific purpose.

Primary keys and foreign keys establish relationships between tables, while validation constraints help prevent invalid data. The use of separate tables also reduces unnecessary data duplication and supports a well-organized database structure.

Therefore, the implemented database provides a foundation for managing the main operations of an online food delivery application. It can be extended further by inserting sample records, executing SQL queries, and developing application features that interact with the database.